EDS 223: Homework Assignment 2

Exploring Patterns of Environmental Justice

Priscilla Pierce

2026-10-22

Exploring Patterns of Environmental Justice

Introduction

This assignment explores how historical redlining continues to shape health and weath outcomes of modern day residents in Los Angeles county and investigates how redlining continues to impact pedestrian and bicycle safety in the region.

Load Packages

```
# Load packages
install.packages("gt")
```

```
Installing package into 'C:/Users/prisc/AppData/Local/R/aarch64-library/4.6'
(as 'lib' is unspecified)
```

```
library(tidyverse)
```

```
── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
✔ dplyr     1.2.1     ✔ readr     2.2.0
✔ forcats   1.0.1     ✔ stringr   1.6.0
✔ ggplot2   4.0.3     ✔ tibble    3.3.1
✔ lubridate 1.9.5     ✔ tidyr     1.3.2
✔ purrr     1.2.2
```

```
── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()    masks stats::lag()
```

```
ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

```
library(sf)
```

```
Linking to GEOS 3.14.1, GDAL 3.12.1, PROJ 9.7.1; sf_use_s2() is TRUE
```

```
library(tmap)
```

```
Registered S3 method overwritten by 'stars':
  method                  from
  st_interpolate_aw.stars sf
```

```
library(here)
```

```
here() starts at C:/Users/prisc/Documents/UCSB/MEDS/
EDS_223_Geospatial_and_Remote_Sensing/eds223-hwk2
```

```
library(gt)
```

```

Attaching package: 'gt'

The following object is masked from 'package:tmap':

    metro
```

Import data

Import EJ Screen data:

```
# Read in geodatabase of EJScreen data at the Census Block Group level and filter
to Los Angeles County
los_angeles <- sf::st_read(here::here("data",
"ejscreen","EJSCREEN_2023_BG_StatePct_with_AS_CNMI_GU_VI.gdb")) |>
  dplyr::filter(CNTY_NAME %in% c("Los Angeles County"))
```

```
Reading layer `EJSCREEN_StatePctiles_with_AS_CNMI_GU_VI' from data source
                                                                        `C:
\Users\prisc\Documents\UCSB\MEDS\EDS_223_Geospatial_and_Remote_Sensing\eds223-
hwk2\data\ejscreen\EJSCREEN_2023_BG_StatePct_with_AS_CNMI_GU_VI.gdb'
  using driver `OpenFileGDB'
Simple feature collection with 243021 features and 223 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -19951910 ymin: -1617130 xmax: 16259830 ymax: 11554350
Projected CRS: WGS 84 / Pseudo-Mercator
```

Import HOLC Redlining data for Los Angeles:

```
# Read in HOLC Redlining data for Los Angeles
redline  <-  sf::st_read(here::here("data",  "mapping-inequality",  "mapping-
inequality-los-angeles.json"))
```

```
Reading layer `mapping-inequality-los-angeles' from data source
                                                                        `C:
\Users\prisc\Documents\UCSB\MEDS\EDS_223_Geospatial_and_Remote_Sensing\eds223-
hwk2\data\mapping-inequality\mapping-inequality-los-angeles.json'
  using driver `GeoJSON'
Simple feature collection with 417 features and 14 fields
Geometry type: MULTIPOLYGON
Dimension:     XY
Bounding box:  xmin: -118.6104 ymin: 33.70563 xmax: -117.7028 ymax: 34.30388
Geodetic CRS:  WGS 84
```

```
# Check CRS of redline against los_angeles
if(st_crs(redline) == st_crs(los_angeles)) {
  print('CRS match')
}else{
  warning('CRS do not match')
  print(paste('redline CRS is', st_crs(redline)))
  print(paste('ej screen CRS is', st_crs(los_angeles)))
}
```

```
Warning: CRS do not match
```

```
[1] "redline CRS is WGS 84"
[2]  "redline  CRS  is  GEOGCRS[\"WGS  84\",\n        DATUM[\"World  Geodetic
```

```
System  1984\",\n                ELLIPSOID[\"WGS  84\",6378137,298.257223563,
\n                   LENGTHUNIT[\"metre\",1]]],\n      PRIMEM[\"Greenwich\",0,
\n          ANGLEUNIT[\"degree\",0.0174532925199433]],\n     CS[ellipsoidal,2],
\n           AXIS[\"geodetic latitude (Lat)\",north,\n               ORDER[1],
\n                                ANGLEUNIT[\"degree\",0.0174532925199433]],\n
AXIS[\"geodetic  longitude  (Lon)\",east,\n                        ORDER[2],\n
ANGLEUNIT[\"degree\",0.0174532925199433]],\n    ID[\"EPSG\",4326]]"
[1] "ej screen CRS is WGS 84 / Pseudo-Mercator"
[2] "ej screen CRS is PROJCRS[\"WGS 84 / Pseudo-Mercator\",\n   BASEGEOGCRS[\"WGS
84\",\n                ENSEMBLE[\"World  Geodetic  System  1984  ensemble\",\n
MEMBER[\"World Geodetic System 1984 (Transit)\"],\n             MEMBER[\"World
Geodetic  System  1984  (G730)\"],\n                    MEMBER[\"World  Geodetic
System  1984  (G873)\"],\n                 MEMBER[\"World Geodetic System 1984
(G1150)\"],\n               MEMBER[\"World Geodetic System 1984 (G1674)\"],\n
MEMBER[\"World Geodetic System 1984 (G1762)\"],\n              MEMBER[\"World
Geodetic System 1984 (G2139)\"],\n             MEMBER[\"World Geodetic System
1984  (G2296)\"],\n                ELLIPSOID[\"WGS  84\",6378137,298.257223563,
\n              LENGTHUNIT[\"metre\",1]],\n          ENSEMBLEACCURACY[2.0]],\n
PRIMEM[\"Greenwich\",0,\n           ANGLEUNIT[\"degree\",0.0174532925199433]],
\n              ID[\"EPSG\",4326]],\n       CONVERSION[\"Popular  Visualisation
Pseudo-Mercator\",\n         METHOD[\"Popular Visualisation Pseudo Mercator\",
\n                   ID[\"EPSG\",1024]],\n            PARAMETER[\"Latitude  of
natural origin\",0,\n              ANGLEUNIT[\"degree\",0.0174532925199433],\n
ID[\"EPSG\",8801]],\n          PARAMETER[\"Longitude of natural origin\",0,\n
ANGLEUNIT[\"degree\",0.0174532925199433],\n                ID[\"EPSG\",8802]],
\n        PARAMETER[\"False easting\",0,\n            LENGTHUNIT[\"metre\",1],
\n               ID[\"EPSG\",8806]],\n         PARAMETER[\"False northing\",0,
\n              LENGTHUNIT[\"metre\",1],\n              ID[\"EPSG\",8807]]],\n
CS[Cartesian,2],\n         AXIS[\"easting (X)\",east,\n              ORDER[1],
\n             LENGTHUNIT[\"metre\",1]],\n        AXIS[\"northing (Y)\",north,
\n             ORDER[2],\n             LENGTHUNIT[\"metre\",1]],\n    USAGE[\n
SCOPE[\"Web mapping and visualisation.\"],\n       AREA[\"World between 85.06°S
and 85.06°N.\"],\n       BBOX[-85.06,-180,85.06,180]],\n    ID[\"EPSG\",3857]]"
```

Match HOLC Redlining data to the coordinate reference system for EJ Screen data:

```
# Change redline CRS to match los_angeles
redline <- st_transform(redline, crs = st_crs(los_angeles))

# Check CRS of redline against los_angeles
if(st_crs(redline) == st_crs(los_angeles)) {
  print('CRS match')
}else{
```

```
  warning('CRS do not match')
  print(paste('redline CRS is', st_crs(redline)))
  print(paste('ej screen CRS is', st_crs(los_angeles)))
}
```

```
[1] "CRS match"
```

Import bike and pedestrian crash report data:

```
# Read in bike and pedestrian crash report data
crash <- read_csv(here::here("data", "SWITRS_LA", "crashes.csv"))
```

```
Rows: 251100 Columns: 80
──                                                                    Column
specification ──────────────────────────────────────────────────────────────
Delimiter: ","
chr  (45): COLLISION_TIME, OFFICER_ID, REPORTING_DISTRICT, CHP_BEAT_TYPE, CI...
dbl  (33): CASE_ID, ACCIDENT_YEAR, JURIS, DAY_OF_WEEK, CHP_SHIFT, POPULATION...
date  (2): PROC_DATE, COLLISION_DATE

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
# Check for NA values in coordinate columns
print(paste('There are', sum(is.na(crash$LATITUDE)), 'NA values in the LATITUDE
column'))
```

```
[1] "There are 97350 NA values in the LATITUDE column"
```

```
print(paste('There  are',  sum(is.na(crash$LONGITUDE)),  'NA  values  in  the
LONGITUDE column'))
```

```
[1] "There are 97350 NA values in the LONGITUDE column"
```

```
print(paste('There are', sum(is.na(crash$POINT_Y)), 'NA values in the POINT_Y
column'))
```

```
[1] "There are 9484 NA values in the POINT_Y column"
```

```
print(paste('There are', sum(is.na(crash$POINT_X)), 'NA values in the POINT_X
column'))
```

```
[1] "There are 9484 NA values in the POINT_X column"
```

Clean latitude and longitude columns and set coordinate reference system to match HOLC Redlining data and EJ Screen data:

```
# Create dataframe of crash columns containing latitude and longitude data
crash_lat_long <- crash[c("LATITUDE", "POINT_Y", "LONGITUDE", "POINT_X")]

# Replace NAs in LATITUDE with POINT_Y values and NAs in LONGITUDE with POINT_X
values
crash_lat_long <- crash_lat_long |>
  mutate(combined_lat = (
              ifelse(is.na(crash_lat_long$LATITUDE),    crash_lat_long$POINT_Y,
crash_lat_long$LATITUDE))) |> # If value in LATITUDE column is NA, replace with
POINT_Y value, otherwise keep LATITUDE value
  mutate(combined_long = (
             ifelse(is.na(crash_lat_long$LONGITUDE),    crash_lat_long$POINT_X,
crash_lat_long$LONGITUDE)
  )) # If value in LONGITUDE column is NA, replace with POINT_X value, otherwise
keep LONGITUDE value

# Replace latitude and longitude columns in crash dataframe with cleaned columns
crash_clean_coords <- crash |>
  mutate(LONGITUDE = crash_lat_long$combined_long,
         LATITUDE = crash_lat_long$combined_lat) |>
  select(c(-POINT_X, -POINT_Y)) |> # remove old POINT_X and POINT_Y columns
  drop_na(LATITUDE, LONGITUDE) |> # drop rows with NA latitude or longitude
   st_as_sf(coords = c("LONGITUDE", "LATITUDE"), crs = 4326) |> # set CRS to
recognize longitude and latitude
  st_transform(crs = st_crs(los_angeles)) # transform CRS to match los_angeles
and redline data

# Check that all CRS match
if((st_crs(redline) == st_crs(crash_clean_coords)) & (st_crs(los_angeles) ==
st_crs(crash_clean_coords))) {
  print('CRS match')
}else{
  warning('CRS do not match')}
```

```
[1] "CRS match"
```

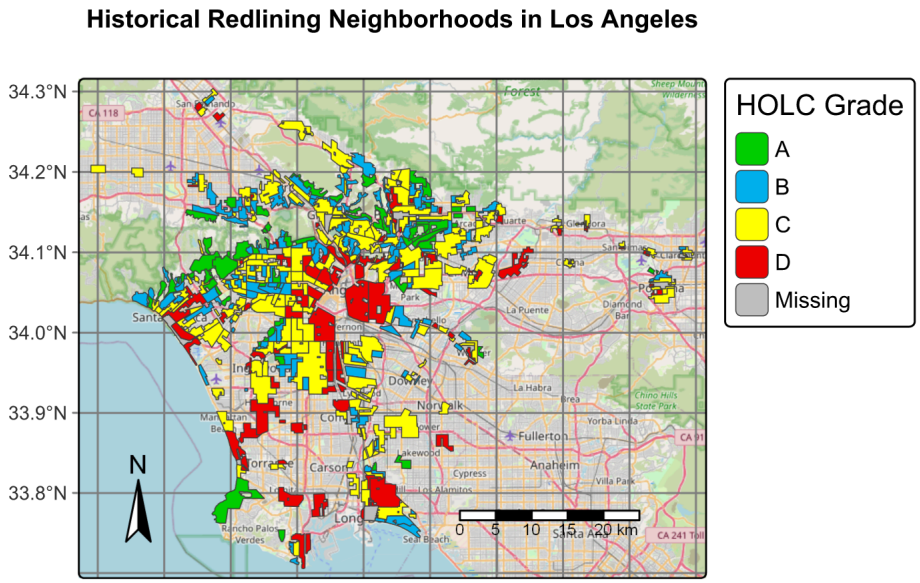
Part 1: Legacy of redlining in current environmental (in)justice

Map of historical redlining neighborhoods in Los Angeles:

```
redline_neighborhoods <- tm_basemap("OpenStreetMap") +
  tm_graticules() +
  tm_shape(redline) + # Add historical redlining data
  tm_polygons(fill = "grade", lwd = 0.5,
              fill.scale = tm_scale_categorical(
                  values = c('green3', 'deepskyblue2', 'yellow1', 'red2')),
              fill.legend = tm_legend(title = "HOLC Grade",
                                      position = tm_pos_out('right'))) +
  tm_compass(position = c('left', 'bottom'), # Add compass
                size = 2) +
    tm_scalebar(position = c('right', 'bottom')) + # Add scalebar
    tm_title('Historical Redlining Neighborhoods in Los Angeles',
              fontface = 'bold')

redline_neighborhoods
```

```
[plot mode] fit legend/component: Some legend items or map compoments do not
fit well, and are therefore rescaled.
ℹ Set the tmap option `component.autoscale = FALSE` to disable rescaling.
```



Of the areas in Los Angeles county that were historically assigned Home Owners' Loan Corporation (HOLC) grades, those that are located in central Los Angeles tended to be ranked lowest (C and D). Many of the areas ranked A or B are located in the hills to the north.

Census block groups and HOLC grades in Los Angeles:

```
# Join redlining data and EJ Screen data
los_angeles_redline <- st_join(los_angeles, redline, join = st_intersects)
```

```
# Summarize percentage of census block groups within each HOLC grade for LA County
HOLC_block_groups <- los_angeles_redline |>
  st_drop_geometry() |> # Drop geometries to convert to standard dataframe
  mutate(Grade = grade) |>
  group_by(Grade) |>
  mutate(Grade = replace_na(Grade, "No Grade Assigned")) |>
  summarize(`Block Count` = n()) |> # Create count column
  mutate(`Percentage of Census Blocks` = (`Block Count` / sum(`Block Count`)) *
100) # Create percentage column

HOLC_block_groups |>
  gt()
```

Grade	Block Count	Percentage of Census Blocks
A	449	4.99555
B	1239	13.78505
C	3058	34.02314
D	1346	14.97552
No Grade Assigned	2896	32.22074

The majority of census blocks in Los Angeles county that were assigned HOLC grades fall within the C (34%) or D grade (15%).

Summarize particulate matter 2.5 percentiles and % low income by HOLC grade:

```
# Summarize Particulate Matter 2.5 percentiles by HOLC grade
HOLC_PM25 <- los_angeles_redline |>
  st_drop_geometry() |> # Drop geometries to convert to standard dataframe
  group_by(grade) |>
  summarize(Mean_PM25_Percentile = mean(P_PM25)) |>  # Calculate mean particulate
matter 2.5 percentiles by grade
  mutate(grade = replace_na(grade, "No Grade Assigned"))

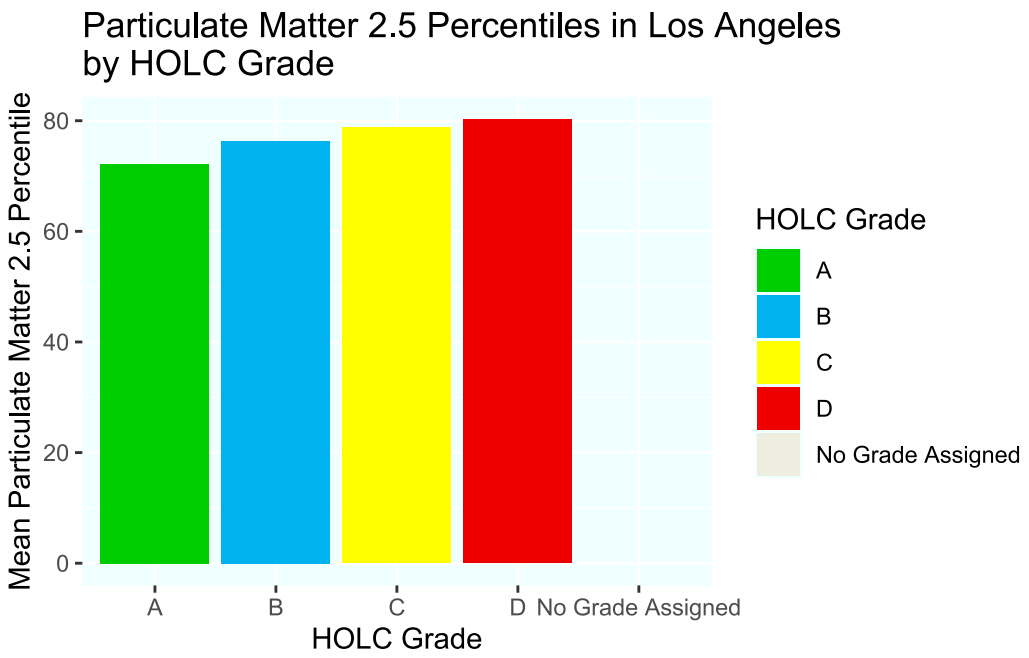
# Summarize % low income by HOLC grade
HOLC_low_income <- los_angeles_redline |>
  st_drop_geometry() |> # Drop geometries to convert to standard dataframe
  group_by(grade) |>
  summarize(Mean_Pct_Low_Income = (mean(LOWINCPCT) * 100)) |> # Calculate mean
% low income residents by HOLC grade
  mutate(grade = replace_na(grade, "No Grade Assigned"))
```

Visualize particulate matter 2.5 percentiles by HOLC grade:

```
holc_colors <- c("A" = 'green3', "B" = 'deepskyblue2', "C" = 'yellow1', "D" =
'red2', "No Grade Assigned" = "ivory2") # Create list of HOLC grade colors

ggplot(HOLC_PM25,  aes(x  =  grade,  y  =  Mean_PM25_Percentile,  fill  =
as.factor(grade))) +
    geom_col() +
     scale_fill_manual(values = holc_colors) + # Assign appropriate colors to
HOLC grades
    labs(
      title = "Particulate Matter 2.5 Percentiles in Los Angeles \nby HOLC Grade",
        x = "HOLC Grade",
```

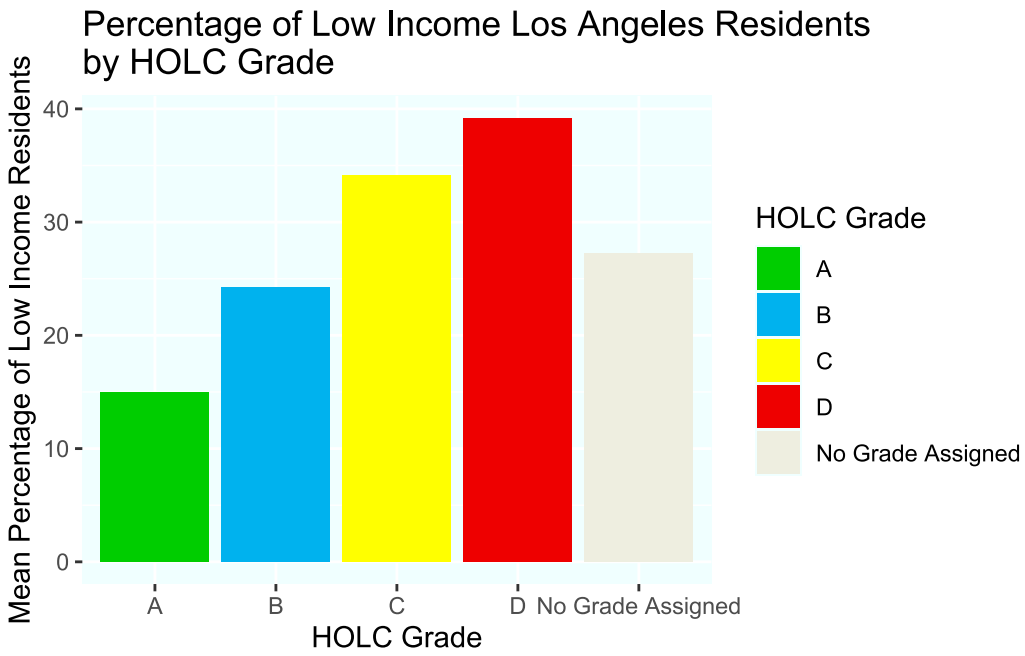
```
        y = "Mean Particulate Matter 2.5 Percentile",
        fill = "HOLC Grade"
    ) +
    theme(panel.background = element_rect(fill = "azure1"))
```



As HOLC grades decrease in ranking from A to D, the mean particulate matter 2.5 percentile increases.

Visualize % low income by HOLC grade:

```
ggplot(HOLC_low_income, aes(x = grade, y = Mean_Pct_Low_Income, fill =
as.factor(grade))) +
    geom_col() +
     scale_fill_manual(values = holc_colors) + # Assign appropriate colors to
HOLC grades
    labs(
       title = "Percentage of Low Income Los Angeles Residents \nby HOLC Grade",
        x = "HOLC Grade",
        y = "Mean Percentage of Low Income Residents",
        fill = "HOLC Grade"
    ) +
    theme(panel.background = element_rect(fill = "azure1"))
```



As HOLC grades decrease in ranking from A to D, the mean percentage of low income residents increases.

Part 1: Reflection

The health and social concequences of historical redlining in Los Angeles county are evident in the above figures. The Historical Redlining Neighborhoods in Los Angeles map shows that areas ranked C or D are largely located in the dense urban center of Los Angeles. Most A and B neigborhoods are located in areas closer to open space and away from the city center.

As HOLC grades decrease in ranking, residents are exposed to poorer air quality as shown by increasing mean particulate matter 2.5 percentiles. The percentage of low income residents also increases as HOLC grades decrease in ranking. The figures show that historical redlining continues to shape the health and weath outcomes of modern day residents.

Part 2: Legacy of redlining in pedestrian and bike accidents

Summarize percent of pedestrian and bicylce accidents in Los Angeles county by HOLC grade

Create a dataframe of pedestrian and bicycle accidents using crashes data:

```
# Filter crashes to include only pedestrian and bicycle accidents
bike_and_ped_accidents <- crash_clean_coords |>
  select(c(PEDESTRIAN_ACCIDENT, BICYCLE_ACCIDENT)) |>
  filter(PEDESTRIAN_ACCIDENT == "Y" | BICYCLE_ACCIDENT == "Y") # Keep rows where
pedestrian or bike accidents are marked Y
```

```
# Check that CRS matches redlining data
if(st_crs(bike_and_ped_accidents) == st_crs(redline)) {
  print('CRS match')
}else{
  warning('CRS do not match')
  print(paste('bike_and_ped_accidents CRS is', st_crs(bike_and_ped_accidents)))
  print(paste('redline CRS is', st_crs(redline)))
}
```

```
[1] "CRS match"
```

Join pedestrian and bicycle data and redline data:

```
# Join bike and pedestrian accident data and redline data
redline_crashes <- st_join(bike_and_ped_accidents, redline)
```

Summarize percent of pedestrian and bicycle accidents by HOLC grade:

```
# Summarize percentage of pedestrian accidents within each HOLC grade
HOLC_ped_accidents <- redline_crashes |>
  st_drop_geometry() |> # Drop geometries to convert to standard dataframe
  select(PEDESTRIAN_ACCIDENT, grade) |>
  drop_na(PEDESTRIAN_ACCIDENT) |> # Remove NA pedestrian accident values
  mutate(Grade = grade) |>
  group_by(Grade) |>
  summarize(`Pedestrian Crashes Count` = n()) |> # Count of pedestrian crashes
by grade
   mutate(`Percentage of Pedestrian Crashes` = (`Pedestrian Crashes Count` /
sum(`Pedestrian Crashes Count`)) * 100) |> # Percentage of pedestrian crashes
by grade
  mutate(Grade = replace_na(Grade, "No Grade Assigned"))

# Summarize percentage of bike accidents within each HOLC grade
HOLC_bike_accidents <- redline_crashes |>
  st_drop_geometry() |> # Drop geometries to convert to standard dataframe
  select(BICYCLE_ACCIDENT, grade) |>
  drop_na(BICYCLE_ACCIDENT) |> # Remove NA bicycle accident values
  mutate(Grade = grade) |>
  group_by(Grade) |>
  summarize(`Bike Crashes Count` = n()) |> # Count of bike crashes by grade
 mutate(`Percentage of Bike Crashes` = (`Bike Crashes Count` / sum(`Bike Crashes
```

```
Count`)) * 100) |> # Percentage of bike crashes by grade
  mutate(Grade = replace_na(Grade, "No Grade Assigned"))

# Join bike and pedestrian crash information by HOLC grade
HOLC_bike_and_ped_accidents <- inner_join(HOLC_bike_accidents,
HOLC_ped_accidents, by = "Grade")

# Add column to joined data to show total bike and pedestrian accidents
HOLC_bike_and_ped_accidents$`Total
Crashes` <- HOLC_bike_and_ped_accidents$`Bike Crashes Count` +
HOLC_bike_and_ped_accidents$`Pedestrian Crashes Count`

# Add column to joined data to show percentages of total bike and pedestrian
accidents
HOLC_bike_and_ped_accidents$`Percentage
of Total Crashes` <- HOLC_bike_and_ped_accidents$`Total Crashes` /
sum(HOLC_bike_and_ped_accidents$`Total Crashes`) * 100

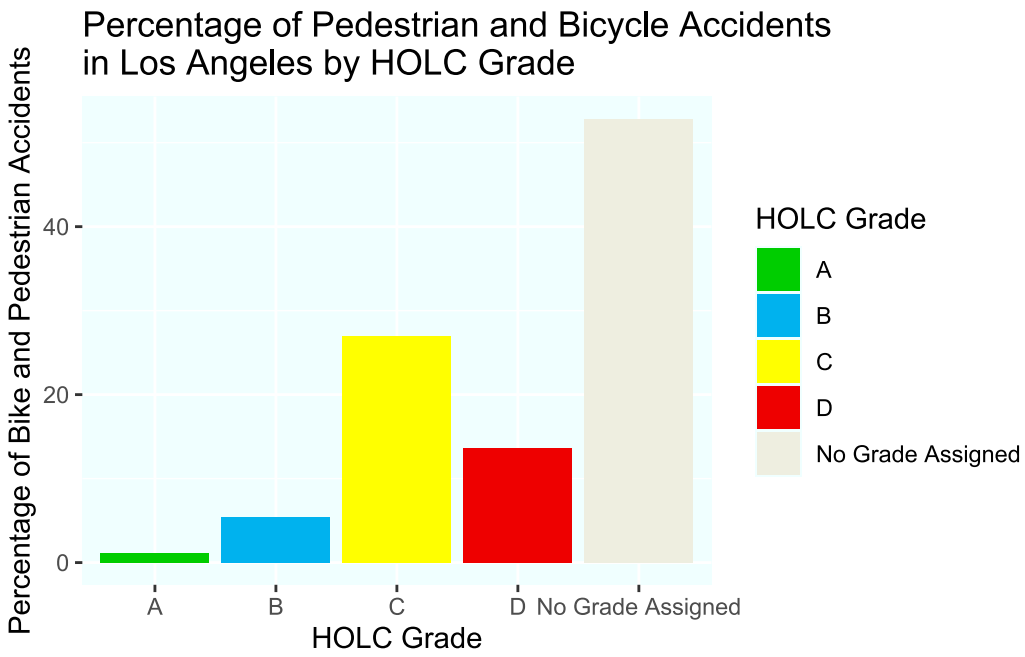
HOLC_bike_and_ped_accidents |>
  gt()
```

Grade	Bike Crashes Count	Percentage of Bike Crashes	Pedestrian Crashes Count	Percentage of Pedestrian Crashes	Total Crashes	Percentage of Total Crashes
A	177	1.442308	194	0.8965708	371	1.094073
B	630	5.133638	1201	5.5504206	1831	5.399587
C	3123	25.448175	6038	27.9046123	9161	27.015630
D	1679	13.681551	2949	13.6288012	4628	13.647891
No Grade Assigned	6663	54.294329	11256	52.0195952	17919	52.842819

Visualize the percentage of pedestrian and bicycle accidents within each HOLC grade

```
ggplot(HOLC_bike_and_ped_accidents, aes(x = Grade, y = `Percentage of Total
Crashes`, fill = as.factor(Grade))) +
    geom_col() +
```

```
     scale_fill_manual(values = holc_colors) + # Assign appropriate colors to
HOLC grades
    labs(
       title = "Percentage of Pedestrian and Bicycle Accidents \nin Los Angeles
by HOLC Grade",
        x = "HOLC Grade",
        y = "Percentage of Bike and Pedestrian Accidents",
        fill = "HOLC Grade"
    ) +
    theme(panel.background = element_rect(fill = "azure1"))
```



Of areas in Los Angeles county that were historically assigned HOLC grades, the largest percentage of bike and pedestrian accidents occurred in areas ranked C (28%) and D (14%).

Data Sources

Data from the United States Environmental Protection Agency's previous EJScreen: Environmental Justice Screening and Mapping Tool, historical redlining data from the University of Richmond Digital Scholarship Lab's Mapping Inequality Project, and University of California, Berkley's Statewide Integrated Traffice Records System data were used to create the map and figures for this assignment.